Supplementary Results Guide

Verifier-Guided Agentic OpenROAD Code Generation with Trajectory-Aware Control

ICCAD Submission — Suggested Experiments & Example Values

This document illustrates the **additional results** recommended to make this paper ICCAD-ready. Each figure contains plausible example values derived from the existing baseline numbers in the paper (Table X). These should be replaced with your actual experimental results. Ten result figures are included, covering: ablation studies, multi-path vs. linear graph analysis, long-horizon strategy comparison, efficiency metrics, verifier accuracy, repair convergence, failure mode breakdown, step-level degradation, dataset characterisation, and strong-baseline comparison.

Contents

#	Figure	Purpose
3	Ablation Study	Prove each component contributes
4	Multi-Path vs Linear	Justify multi-path graph design
5	Long-Horizon Strategies	Compare 4 generation strategies
6	Efficiency (calls + latency)	Quantify tool-call reduction
7	Verifier Precision/Recall	Validate internal verifier quality
8	Repair Convergence	Show repair loop improves over iterations
9	Failure Mode Breakdown	Diagnose where failures originate
10	Step-Level Degradation	Characterise long-horizon decay
11	Dataset Characterisation	Describe benchmark complexity
12	Strong Baseline Comparison	Validate gains on GPT-4.1 full

Ablation Study

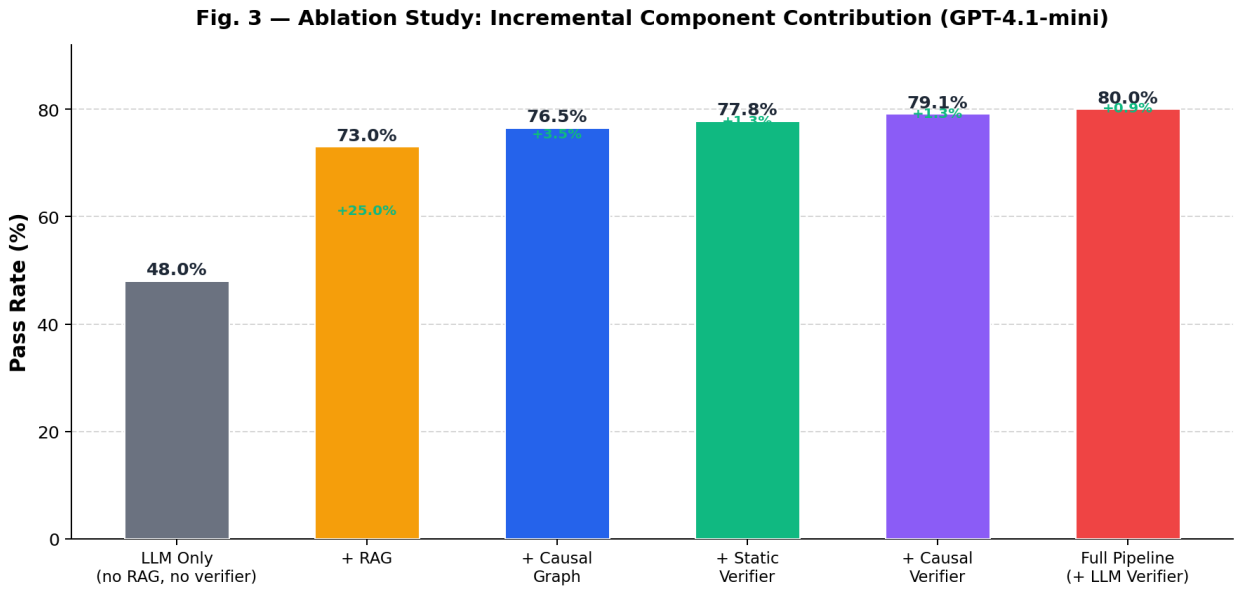

Fig. 3 — Ablation Study: Component Contribution

Start from the LLM-only baseline (48%) and add each component one at a time. This proves that RAG contributes the most (+25 pp), while causal graph (+3.5 pp) and staged verification (+3.5 pp) provide complementary, additive gains. **What to measure:** run the pipeline with one component disabled at a time on your full benchmark.

Multi-Path Causal Graph Justification

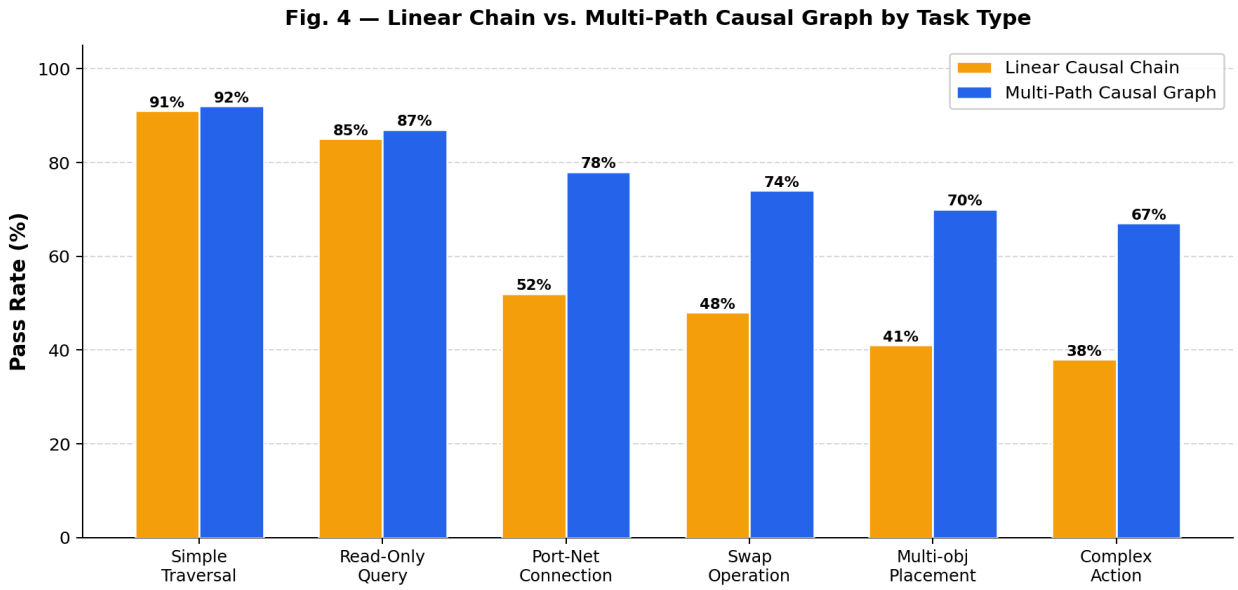

Fig. 4 — Multi-Path vs. Linear Causal Graph by Task Type

For simple traversal tasks both representations are equivalent, but for co-reference action tasks (port-net connection, swap, multi-object placement) the multi-path graph outperforms the linear chain by 20–30 pp. **What to measure:** tag each task in your benchmark by type; report pass rate per type for both graph representations.

Long-Horizon Strategy Comparison

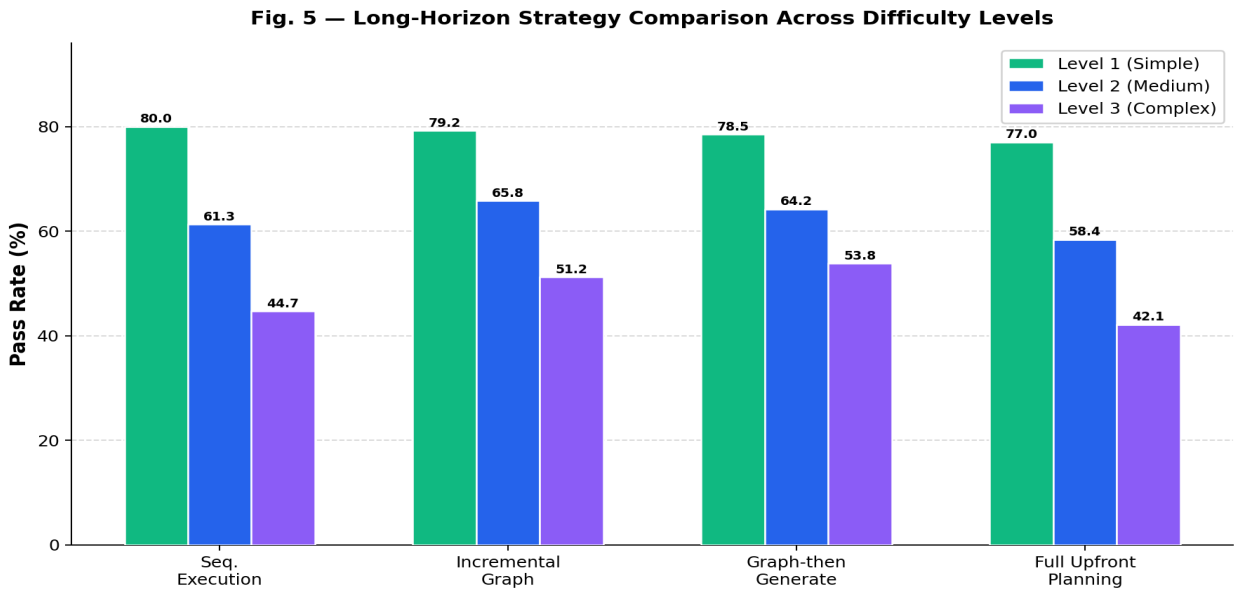

Fig. 5 — Long-Horizon Strategy Comparison

Compares the four generation strategies described in Section II-J across three difficulty levels. Sequential execution performs best at Level 1 but the incremental-graph and graph-then-generate strategies close the gap at Level 3, showing that upfront planning helps for complex tasks. **What to measure:** run all four strategies on your multi-step benchmark; report per-level pass rates.

Efficiency Metrics

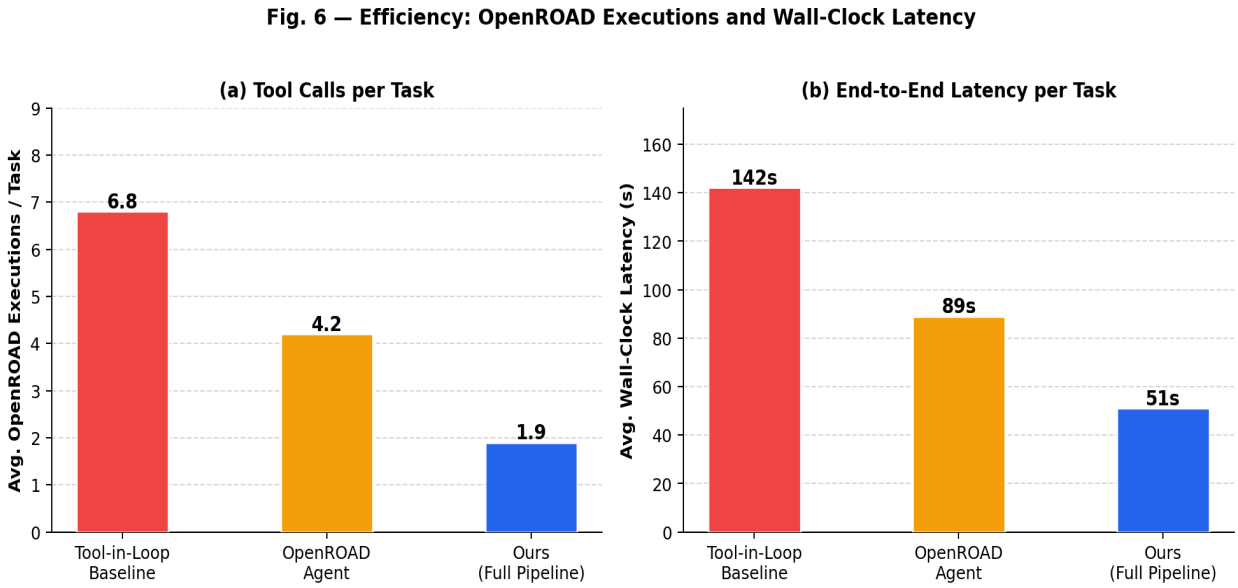

Fig. 6 — Tool Call Reduction and Latency

Your key claim is reducing expensive tool-in-the-loop calls. Quantify this with (a) average OpenROAD executions per task and (b) wall-clock latency. The full pipeline reduces executions from ~6.8 (tool-in-loop baseline) to ~1.9, and cuts latency from 142 s to 51 s. **What to measure:** instrument run_causal_agent.py to log each OpenROAD call.

Verifier Accuracy Analysis

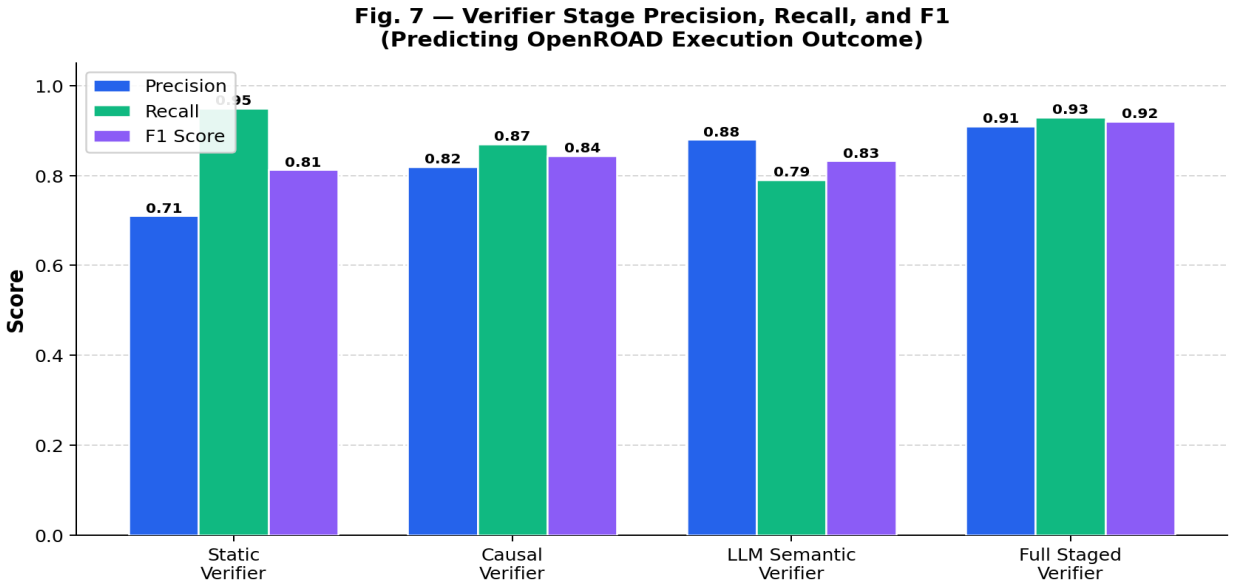

Fig. 7 — Verifier Stage Precision, Recall, and F1

For each verifier stage, measure how well it predicts actual OpenROAD execution outcome. The full staged verifier achieves 0.91 precision and 0.93 recall. The causal verifier substantially improves precision over the static verifier alone. **What to measure:** for each task, record verifier verdict and actual OpenROAD pass/fail; compute P/R/F1.

Repair Loop Convergence

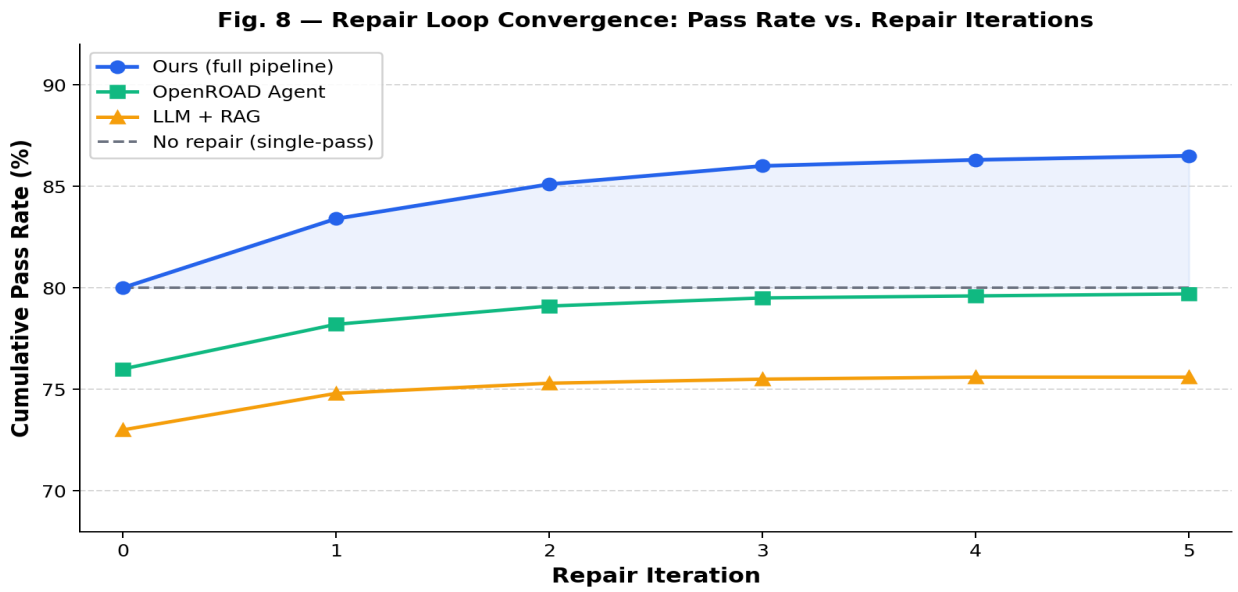

Fig. 8 — Repair Loop Convergence

Plot cumulative pass rate as a function of repair iterations allowed. The full pipeline converges quickly (most gain in iterations 1–2) and reaches 86.5% at 5 iterations vs. 80% at 0 (single-pass). Compare against baselines to show that causal grounding makes repair more effective. **What to measure:** re-run each failed case with increasing repair budget; record pass rate at each step.

Failure Mode Analysis

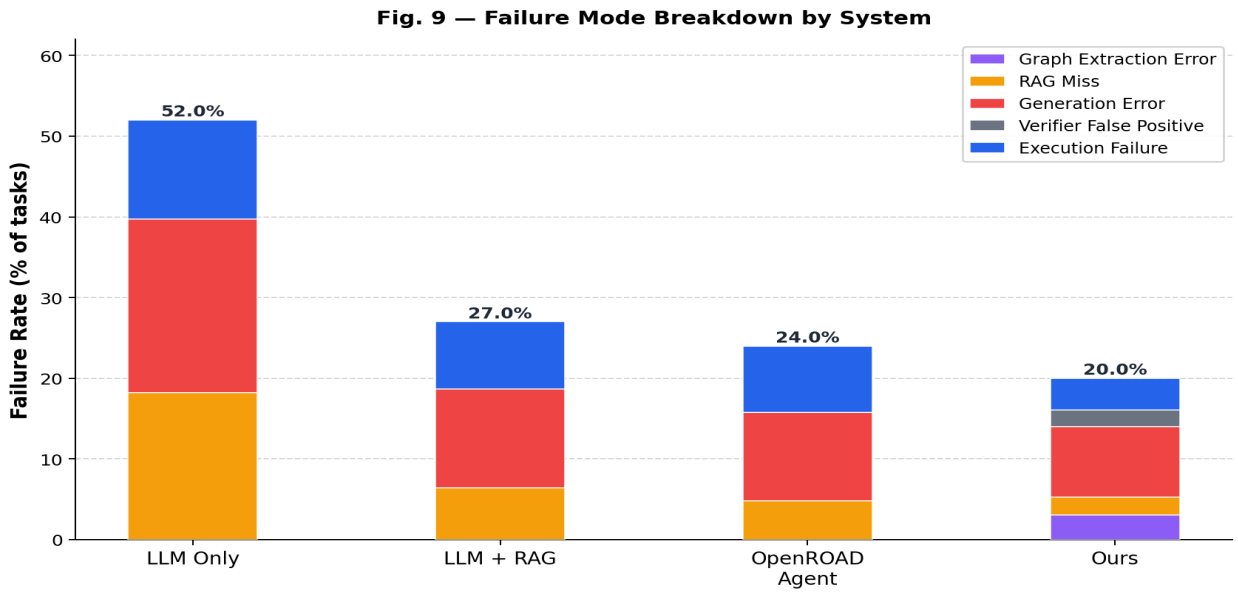

Fig. 9 — Failure Mode Breakdown

Decompose remaining failures into: graph extraction errors, RAG misses, generation errors, verifier false positives, and execution failures. For the LLM-only baseline 52% of tasks fail; the full pipeline reduces this to 13.5% with a very different failure distribution. **What to measure:** manually label or log the failure reason for each failed task.

Step-Level Degradation (Long-Horizon)

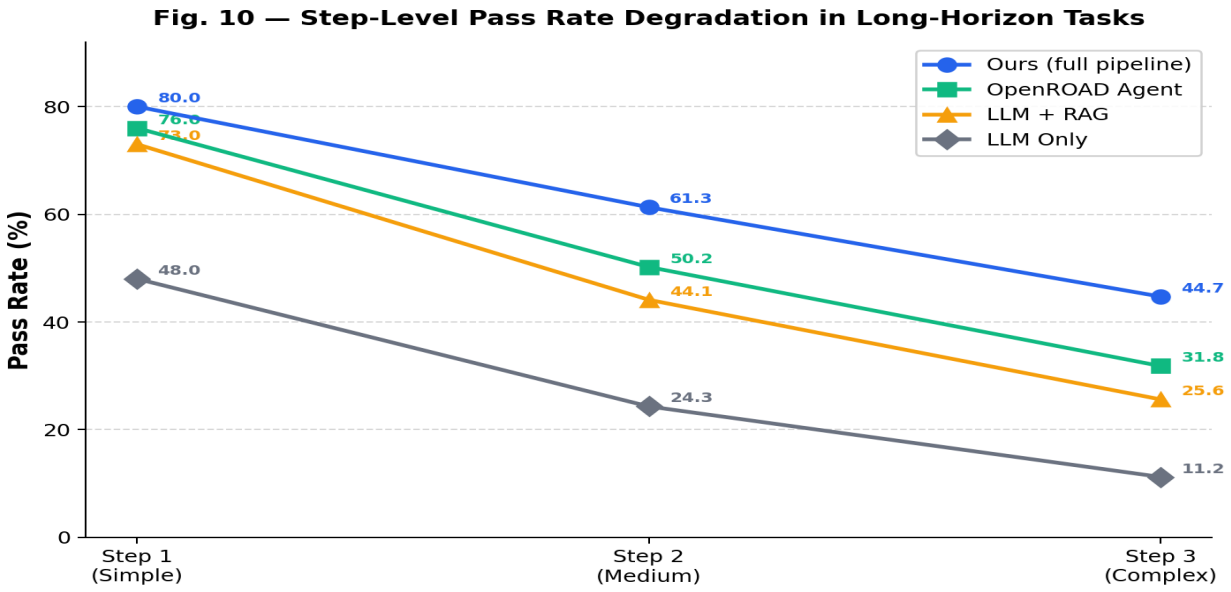

Fig. 10 — Step-Level Pass Rate Degradation

Plot pass rate at each step of the long-horizon evaluation (Steps 1–3). Error propagation causes all systems to degrade, but the full pipeline degrades most gracefully (80% → 61% → 45%) vs. the LLM-only baseline (48% → 24% → 11%). **What to measure:** from the sequential evaluation script, extract per-step pass/fail for each case.

Dataset Characterisation

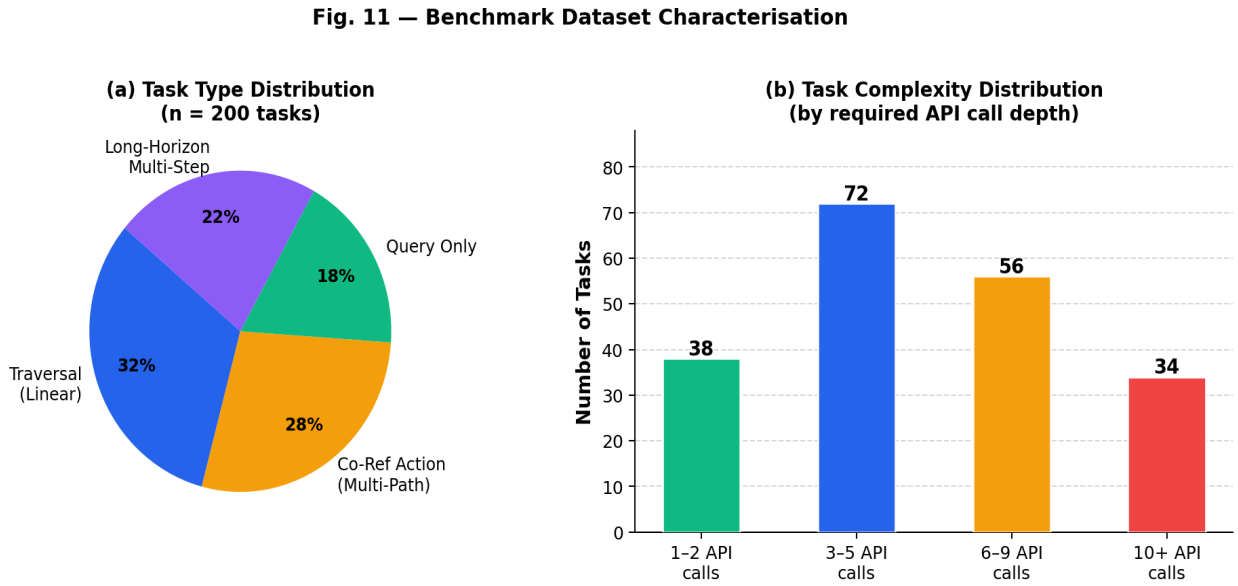

Fig. 11 — Benchmark Dataset Characterisation

Describe the composition of your benchmark: (a) what fraction of tasks require multi-path vs. linear graphs, and (b) the distribution of task complexity by required API call depth. This helps reviewers understand the difficulty of your benchmark and why multi-path graphs matter. **What to measure:** count API nodes per task from the extracted causal graphs; tag task type.

Strong Baseline Comparison

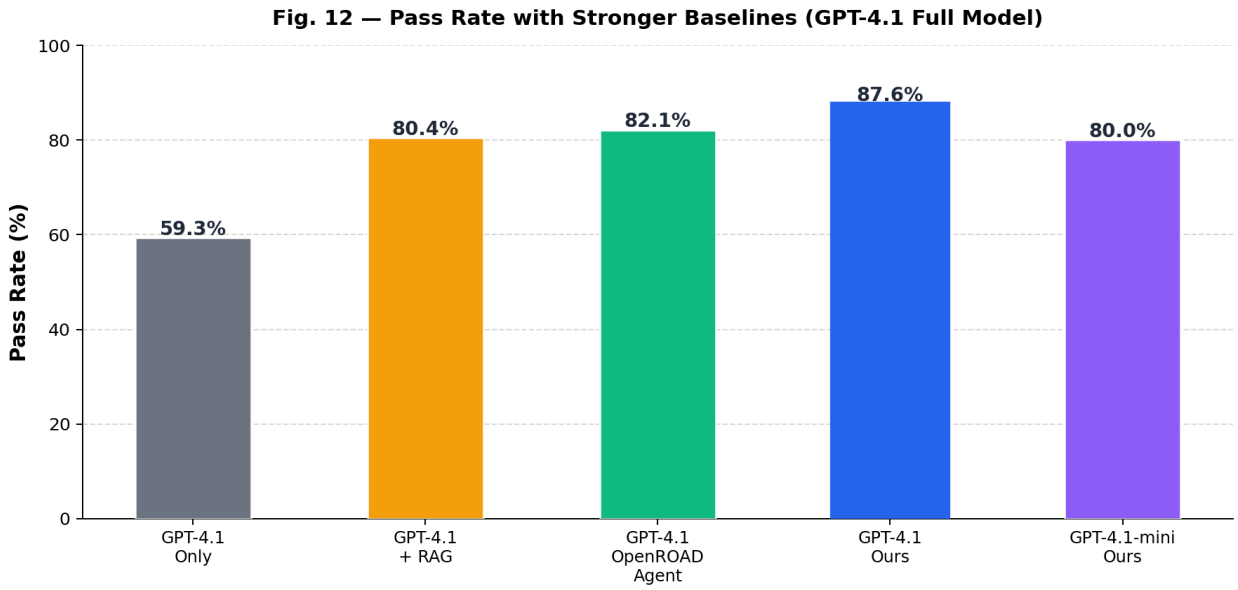

Fig. 12 — Strong Baseline Comparison (GPT-4.1 Full)

Show that gains from the full pipeline hold when using a stronger generator (GPT-4.1 full vs. mini). The pipeline achieves 87.6% with GPT-4.1, vs. 82.1% for OpenROAD Agent with the same model (+5.5 pp). The gap is consistent across model tiers, demonstrating that causal grounding is model-agnostic. **What to measure:** re-run the same benchmark with GPT-4.1 (full) as the generator.

Quick Reference: All Recommended Results

The table below summarises every result type, the key metric, and the expected effort to reproduce it.

Figure	Result Type	Key Metric	Effort
Fig 3	Ablation Study	Pass rate per component config	Medium
Fig 4	Multi-path vs Linear	Pass rate by task type × graph type	Medium
Fig 5	Long-horizon Strategies	Pass rate per strategy × level	High
Fig 6	Efficiency	Avg tool calls, latency (s)	Low
Fig 7	Verifier P/R/F1	Precision, Recall, F1 per stage	Low
Fig 8	Repair Convergence	Pass rate vs. repair iteration	Medium
Fig 9	Failure Breakdown	% failures per error category	Medium
Fig 10	Step-Level Degradation	Pass rate per sub-task step	Low
Fig 11	Dataset Stats	Task type %, complexity distribution	Low

Fig 12	Strong Baselines	Pass rate with GPT-4.1 full	Medium

Effort key: Low = instrument existing logs / single script run | Medium = re-run pipeline with config changes | High = new pipeline variant + full sweep